本章的問題

給定 $A \in \mathbb{R}^{m \times n}$、$b \in \mathbb{R}^m$，研究

$$Ax = b.$$

需要分開回答：

1. 是否存在解？
2. 解是否唯一？
3. 如何用有限步驟求出所有解？
4. 計算量與數值風險是什麼？

本章直接處理前 3 問。第 11 章會加入有限精度、穩定性與條件數。

不要把「消去後無矛盾」誤寫成「有唯一解」。存在與唯一是兩個層次。

本章符號、體與 row/column 用語

- m：輸出空間維度，等於方程數與 A 的 row 數。
- n：輸入空間維度，等於未知數與 A 的 column 數。
- A_{ij}：第 i row、第 j column 的元素。
- e_i：第 i 個標準 basis vector。
- P：permutation matrix（排列矩陣）。
- $\mathbb{R}$ 或 $\mathbb{C}$：本階段矩陣元素所在的 field（體）。

若 A 是 $m \times n$，x 必須是 $n \times 1$，b 必須是 $m \times 1$。任何乘法或加法前先檢查這些形狀。

中文教材對 row 與 column 的翻譯不統一，本文固定：

- row（列）：水平方向；
- column（欄）：垂直方向。

有些書把 column 寫成「行」，因此看到中文「行」時必須從上下文確認。本文保留英文 row/column 於容易混淆處，所有矩陣運算都以這裡的定義為準。

維度不匹配是可先排除的錯誤，不需要進到數值計算才發現。

兩種同時成立的圖像

考慮

$$x + 3y = 7, \qquad 2x - y = 0.$$

Row picture：兩個方程分別是 xy 平面的直線，交點是 $(x, y) = (1, 2)$。

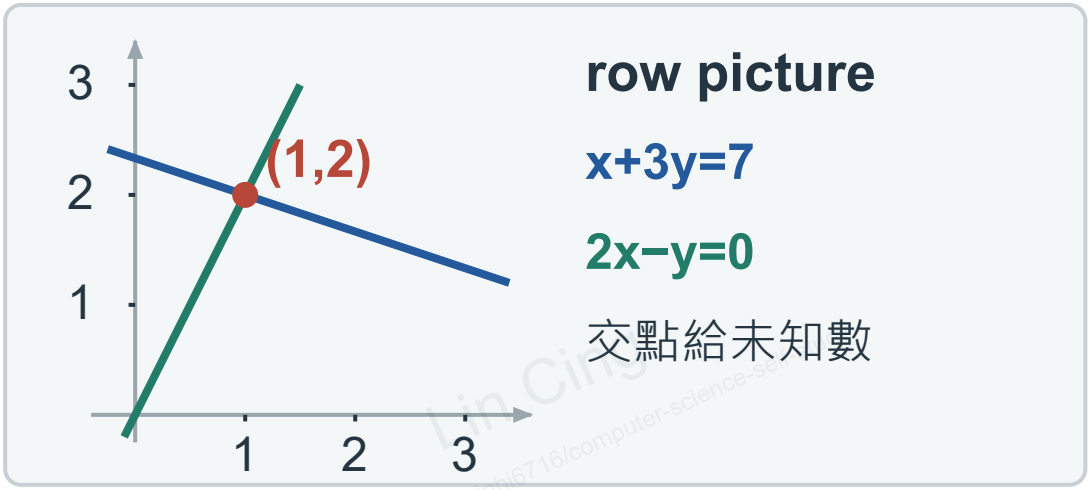
Column picture：

$$x \begin{bmatrix} 1 \\ 2 \end{bmatrix} + y \begin{bmatrix} 3 \\ -1 \end{bmatrix} = \begin{bmatrix} 7 \\ 0 \end{bmatrix}.$$

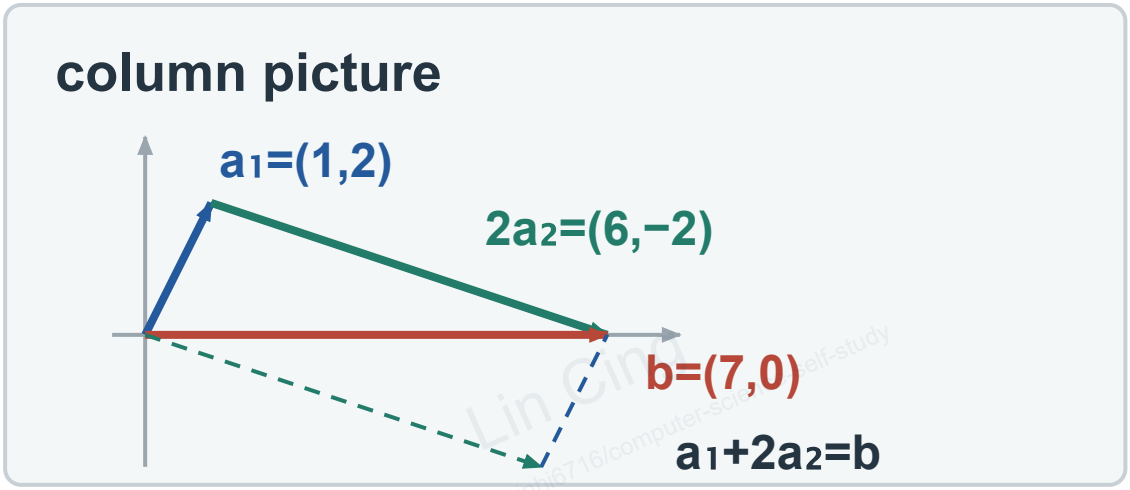
係數 $(1, 2)$ 使欄向量組合成目標。這兩張圖都在描述同一組數字，但凸顯不同結構：row view 看限制，column view 看可達方向。

第三維或更多方程時，row picture 由 lines 變成 planes 或 hyperplanes，column view 仍保留「係數組合」。

Row 觀點：兩條方程的交點



Column 觀點：組合成目標向量



存在性與唯一性

方程組可能有：

- 唯一解：pivot 覆蓋每個未知數且沒有矛盾列。
- 無限多解：一致，但至少一個 free variable。
- 無解：出現 $[0 \ \cdots \ 0 \mid c]$，其中 $c \neq 0$。

對 $A \in \mathbb{R}^{m \times n}$：

- $m < n$ 時，齊次方程 $Ax = 0$ 必有非零解，但 $Ax = b$ 仍可能無解。
- 每個 $b \in \mathbb{R}^m$ 都能解，等價於 column space（欄空間）是 $\mathbb{R}^m$。
- 對每個可解的 b 解都唯一，等價於 nullspace（零空間）只有零向量。

rank-nullity 會在下一章正式使用；此刻可用 free variable 的數目理解。

例子中的完整判定

解

$$\begin{aligned} x &+ 2y &&= 4, \\ 2x &+ 4y &&= 9. \end{aligned}$$

第二列減去第一列的 2 倍，得到

$$0x + 0y = 1.$$

這是矛盾列，因此無解。Column view 中，目標 $(4, 9)^T$ 不在 $\text{span}\{(1, 2)^T, (2, 4)^T\}$，該 span 只有直線 $v_2 = 2v_1$。

若右端改為 $(4, 8)^T$，消去後得到 $0 = 0$。限制只剩一條，解是

$$x + 2y = 4 \quad \Rightarrow \quad (x, y) = (4, 0) + t(-2, 1),\ t \in \mathbb{R}.$$

free variable 不是計算殘餘，而是真正描述解集方向的parameter。

Gaussian elimination（高斯消去）

消去法用可逆 row operations（列運算）保持解集：

1. 交換兩列。
2. 將一列乘非零純量。
3. 將某列的純量倍加到另一列。

從第一欄選非零 pivot（主元），以它清除該欄下方元素。接著移到右側下一欄，直到無列可處理。

示例：

$$\left[\begin{array}{ccc|c} 1 & 2 & 1 & 8 \\ 3 & 8 & 4 & 24 \\ 0 & 4 & 3 & 17 \end{array}\right]$$

先用 row 1 清除 row 2 的第一欄，再處理第二欄。每一步都不改變 equations 的解集。

row operation 是在方程左右的同時運算；不是任意改動單一資料格。

pivot 與 multiplier 的意義

若使用第 k 個 pivot p 清除位置 (i,k)，multiplier（乘數）是

$$\ell_{ik} = \frac{A_{ik}}{p},$$

並執行 $R_i \leftarrow R_i - \ell_{ik} R_k$。

前頁例子第一欄的 multiplier 是 $3/1 = 3$，消去後得

$$\begin{bmatrix} 1 & 2 & 1 & | & 8 \\ 0 & 2 & 1 & | & 0 \\ 0 & 4 & 3 & | & 17 \end{bmatrix}.$$

接著第二欄 pivot 為 2，multiplier 為 $4/2 = 2$。

小 pivot 在精確算術中還可用；有限精度時可能放大誤差，第 11 章會用 partial pivoting 處理。

回代與 rank 的初步

前頁完成消去：

$$\begin{bmatrix} 1 & 2 & 1 & | & 8 \\ 0 & 2 & 1 & | & 0 \\ 0 & 0 & 1 & | & 17 \end{bmatrix}.$$

由下而上：

$$z = 17, \quad 2y + 17 = 0 \Rightarrow y = -17/2,$$
$$x + 2(-17/2) + 17 = 8 \Rightarrow x = 8.$$

有三個非零 pivots，方陣 rank 為 3，因此解唯一。若有 zero row 且右端相同為 zero，那一列不提供新限制；若有 rank-deficient 欄，會產生自由變數。

rank 等於非零 pivot 數目。一般列階梯形已可辨認 pivot column 的索引；取基底時應取原矩陣中對應的欄，而不是消去後改變的欄。

消去流程的完整決策順序

對 $A \in \mathbb{F}^{m \times n}$ 與 $b \in \mathbb{F}^m$，流程如下：

1. 固定體與未知數順序。精確算術可選任一非零 pivot；有限精度通常選目前欄中絕對值最大的元素作 pivot。
2. 從左到右掃描 columns。若目前 column 在 pivot row 以下全為零，跳過該 column；它對應的未知數通常成為 free variable。
3. 若找到非零元素，交換它到 pivot row，消去下方同 column 的元素，再向右移動。每一步左乘可逆初等矩陣。
4. 完成 row echelon form 後，先找 $[0 \cdots 0 \mid c]$。若有 $c \neq 0$，系統無解；若所有矛盾列都不存在，系統一致。
5. 求零空間時，必須另解齊次系統：保留同一個階梯形係數矩陣，把右端全部改為 0。對每個 free column，設對應 free variable 為 1、其餘 free variables 為 0，再回代 pivot variables，得到 $As_j = 0$ 的 special solutions。不能沿用非齊次右端，否則得到的是原系統的解，而非零空間向量。
6. 為求一個 particular solution，令全部 free variables 為 0，回代 pivot variables；完整解為

$$x = x_p + \sum_{\text{free } j} t_j s_j.$$

消去流程的完整決策順序 （續）

這個流程也需要「在指定體上」的前提。若 pivot 在某個體中為零，不能只因它在另一個體中非零就沿用同一消去；檢查 consistency 時也必須同步套用 row operations 到 augmented column。

例如 $x_1 + 2x_2 = 4$：特解為 $x_p = (4, 0)^T$；齊次式 $x_1 + 2x_2 = 0$ 在 $x_2 = 1$ 時給 $s = (-2, 1)^T$，故全解 $x = (4, 0)^T + t(-2, 1)^T$。若錯用原右端，會得到 $(2, 1)^T$，其像為 4，並不在零空間中。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

用矩陣乘積執行列運算

若要把第二列加上第一列的 2 倍，對單位矩陣做此運算得到：

$$E = \begin{bmatrix} 1 & 0 \\ 2 & 1 \end{bmatrix}, \qquad EI_2 = \begin{bmatrix} 1 & 0 \\ 2 & 1 \end{bmatrix}.$$

左乘 E 對 rows 作用，右乘 E 對 columns 作用。這是一個關鍵的側別：

$$EA \text{ 改 rows}, \qquad AF \text{ 改 columns}.$$

初等矩陣 E 可逆，且逆矩陣撤銷相應 row operation。例如上面 E 的逆是

$$E^{-1} = \begin{bmatrix} 1 & 0 \\ -2 & 1 \end{bmatrix}.$$

題目若要求先乘哪個運算，符號順序不能對調。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

消去矩陣的乘法順序

若先施加 E_1，再施加 E_2，最後矩陣是

$$E_2(E_1A) = (E_2E_1)A.$$

Left multiplication 的順序是由右往左對應時間，但矩陣本身先合併成 E_2E_1。若交換 E_1, E_2，row operations 的結果可能不同。

若對方陣不換列、也不縮放 pivot row，只用下方列減去 pivot row 的倍數，從多步消去得到

$$E_k \cdots E_2E_1A = U.$$

因此

$$A = (E_k \cdots E_1)^{-1}U = LU.$$

這個關係將在「從消去到 $A = LU$」整理為 LU 分解。第一欄消去所得的 ℓ_{ik} 將直接進入 L。

正確公式是 $(E_k \cdots E_1)^{-1} = E_1^{-1} \cdots E_k^{-1}$；逆矩陣的乘積必須反轉順序。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

何時需要 row swap

若第一欄 pivot 位置為零，可以把下方非零元素交換上來：

$$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} b \\ a \end{bmatrix}.$$

最後一列若為

$$[0 \ \cdots \ 0 \mid c],$$

且 $c \neq 0$，方程組無解。若 row exchange 能把非零項帶入 pivot 位置，便繼續消去；若整欄下方皆零，就必須跳到下一欄。

pivot 位置並非永遠在對角線；對矩形矩陣尤其必須依欄搜尋。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

矩陣乘法與 dimension

若 $A \in \mathbb{F}^{m \times n}$、$B \in \mathbb{F}^{n \times p}$，則

$$(AB)_{ij} = \sum_{k=1}^{n} A_{ik} B_{kj}, \qquad AB \in \mathbb{F}^{m \times p}.$$

中間維度 n 必須相同；輸出維度由左矩陣 rows、右矩陣 columns 決定。

「AB」只有在上述形狀符合時才存在。即使 AB 存在，BA 也可能沒有定義。即使兩者都存在，通常

$$AB \neq BA.$$

矩陣乘法代表先施加右側轉換、再施加左側轉換；不交換正好反映作用順序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Column 與 row 組合觀點

$$AB = A\begin{bmatrix} b_1 & \cdots & b_p \end{bmatrix} = \begin{bmatrix} Ab_1 & \cdots & Ab_p \end{bmatrix}.$$

所以 AB 的每一欄是 A 各欄依 B 對應欄係數的組合。也能按 rows 分解：

$$AB = \begin{bmatrix} a_1^T \\ \vdots \\ a_m^T \end{bmatrix} B = \begin{bmatrix} a_1^T B \\ \vdots \\ a_m^T B \end{bmatrix}.$$

例：

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} = \begin{bmatrix} 1 & -2 \\ 3 & -4 \end{bmatrix},$$

這是右乘對角符號矩陣，只將第二欄乘以 -1，沒有交換欄位置。

block multiplication 合法的前提是每個 block 介面維度一致。

不可交換與分配律

取

$$A = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}, \quad B = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}.$$

計算得

$$AB = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}, \qquad BA = \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix}.$$

所以 $AB \neq BA$。可用的律包括

$$A(B+C) = AB + AC, \quad (A+B)C = AC + BC,$$

以及當尺寸配合時 $A(BC) = (AB)C$。若 A 可逆，可左消去 $AB = AC$ 得 $B = C$；純量加法消去不可直接套用到一般矩陣乘法。

不要把實數的 commutative law 自動套到矩陣。

兩個尺寸不同的乘積，trace 為何相同？

若 $A \in \mathbb{R}^{m\times n}$、$B \in \mathbb{R}^{n\times m}$，則 AB 是 $m \times m$，BA 是 $n \times n$。即使尺寸不同，兩者都是方陣，因此都可計算 trace（對角元素之和）。

$$\begin{aligned}\operatorname{tr}(AB) &= \sum_{i=1}^{m}\sum_{j=1}^{n} a_{ij}b_{ji} \\ &= \sum_{j=1}^{n}\sum_{i=1}^{m} b_{ji}a_{ij} = \operatorname{tr}(BA).\end{aligned}$$

這裡交換的是有限和的順序，以及**純量**乘積的次序；沒有假設 $AB = BA$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

用同一組數值檢查 trace

$$A = \begin{bmatrix}1 & 2 & 0\\ 0 & 1 & 3\end{bmatrix}, \quad B = \begin{bmatrix}2 & 1\\ 1 & 0\\ 0 & -1\end{bmatrix}.$$

$$AB = \begin{bmatrix}4 & 1\\ 1 & -3\end{bmatrix}, \qquad BA = \begin{bmatrix}2 & 5 & 3\\ 1 & 2 & 0\\ 0 & -1 & -3\end{bmatrix}.$$

因此 $\operatorname{tr}(AB) = 4 - 3 = 1$，而 $\operatorname{tr}(BA) = 2 + 2 - 3 = 1$。兩個矩陣的尺寸甚至不同，不能因此把它們當成同一矩陣。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

六個乘積的兩種加總

$a_{ij}\,b_{ji}$	j=1	j=2	j=3	AB diagonal
i=1	1·2 = 2	2·1 = 2	0·0 = 0	4
i=2	0·1 = 0	1·0 = 0	3·(−1) = −3	−3
BA diagonal	2	2	−3	**sum = 1**

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

循環移位不是任意重排

尺寸允許時，$\operatorname{tr}(ABC)=\operatorname{tr}(BCA)=\operatorname{tr}(CAB)$：把 BC 視為一個矩陣即可套用上一頁的等式。一般不能任意交換相鄰因子。

例如令

$$P=\begin{bmatrix}0&1\\0&0\end{bmatrix},\quad Q=\begin{bmatrix}0&0\\1&0\end{bmatrix},\quad R=\begin{bmatrix}1&0\\0&0\end{bmatrix}.$$

由 $PQ=R$、$QP=\operatorname{diag}(0,1)$，得 $\operatorname{tr}(PQR)=1$，但 $\operatorname{tr}(QPR)=0$。循環移位保留相對順序；交換 P,Q 則沒有。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

trace 與交換子：先說明使用的體

對實數或複數上的 $n \times n$ 矩陣，

$$\operatorname{tr}(AB - BA) = 0 \neq n = \operatorname{tr}(I_n),$$

因此 $AB - BA = I_n$ 不可能成立。這個結論利用 $n \neq 0$ 在所用的體中成立。

在特徵為 2 的體上，取上一頁的 P, Q，有 $PQ - QP = I_2$：因為減號與加號相同，且 $\operatorname{tr}(I_2) = 1 + 1 = 0$。所以「不可能」不能不加條件推廣到任意體。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

逆矩陣的定義

對 $A \in \mathbb{F}^{n \times n}$，若存在 B 使

$$AB = BA = I_n,$$

則 A 可逆，並寫 $B = A^{-1}$。方陣是為避免 left inverse 與 right inverse 混淆，但即使形狀相同，仍須兩側都檢查定義。

逆矩陣若存在則唯一。若 $AB = I$ 且 $CA = I$，則

$$B = IB = (CA)B = C(AB) = CI = C.$$

因此左右逆在方陣可逆情況下沒有不同。

第 2 章以方陣為主；長方矩陣的 one-sided inverse 需要額外假設。

若 $A \in \mathbb{R}^{m \times n}$ 且 $m \neq n$，不能同時有 $AB = I_m$、$BA = I_n$ 這組雙邊逆。滿 column rank $n \leq m$ 時，$A^T A$ 可逆，

$$A_L = (A^T A)^{-1} A^T, \qquad A_L A = I_n,$$

稱為 left inverse，但不是 A 的雙邊 inverse，且可能有其他 left inverses。滿 row rank $m \leq n$ 時，對稱地有

$$A_R = A^T (A A^T)^{-1}, \qquad A A_R = I_m.$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

逆矩陣的定義 （續）

兩個公式都把 dimension 與 rank 條件放在前面；行列式只對方陣可談，不能用它把長方矩陣直接判成可逆。若 rank 不足，需要 Moore–Penrose pseudoinverse 或最小平方模型，而不是硬套上述 inverse。

消去時只對 rows 做 row operations 可直接保留同一個 x；若在 columns 上做運算，實際上是在重排或混合未知數，必須同步改變 variable ordering。這是解方程式時只用 row operations 的關鍵原因。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Gauss–Jordan 求逆

要對 A 求逆，把 A 與 I 並排：

$$[A \mid I],$$

只做 row operations，直到左側成為 I：

$$[I \mid A^{-1}].$$

例：

$$\left[\begin{array}{cc|cc} 2 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 \end{array}\right] \xrightarrow{R_1 \leftarrow R_1 - R_2} \left[\begin{array}{cc|cc} 1 & 0 & 1 & -1 \\ 1 & 1 & 0 & 1 \end{array}\right] \xrightarrow{R_2 \leftarrow R_2 - R_1} \left[\begin{array}{cc|cc} 1 & 0 & 1 & -1 \\ 0 & 1 & -1 & 2 \end{array}\right].$$

所以 $A^{-1} = \begin{bmatrix} 1 & -1 \\ -1 & 2 \end{bmatrix}$。

這個結果可用 $AA^{-1} = I$ 驗算；初等 rows 若造成左側 zero row，則 A 不可逆。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

可逆性的等價條件

對方陣 $A \in \mathbb{F}^{n \times n}$，下列命題在指定體上等價：

1. $Ax = 0$ 只有 $x = 0$。
2. A^{-1} 存在。
3. A 的 n 個 columns 線性獨立。
4. A 的 columns span $\mathbb{F}^n$。
5. elimination 得到 n 個非零 pivots。
6. $\det A \neq 0$。

前五項可由消去證明：零空間只有零解等價於沒有自由變數，方陣因而有 n 個主元，每個右端皆可唯一回代；逐一以標準基底為右端求解，即可組成逆矩陣。反過來，左乘逆矩陣立刻給出齊次方程只有零解。第 6 項需先定義 determinant，將於第 5 章完整處理。矩形矩陣不可照搬全部條件。

命題中的「在指定體上」有實質作用。例如某些矩陣在實數體上可逆，在有限體上可能不可逆。

從消去到 $A = LU$

若不需 row exchange，且 Gaussian elimination 的 pivots 不為零，則

$$A = LU,$$

其中 L 是 unit lower triangular，U 是 upper triangular。L_{ik} 記下第 i 列減去第 k 列的乘數。

例：

$$A = \begin{bmatrix} 2 & 1 \\ 4 & 3 \end{bmatrix}.$$

第一欄 multiplier 是 $4/2 = 2$：

$$L = \begin{bmatrix} 1 & 0 \\ 2 & 1 \end{bmatrix}, \quad U = \begin{bmatrix} 2 & 1 \\ 0 & 1 \end{bmatrix}.$$

檢查 $LU = A$。對同一個 A 解多個右端 b，分解後只需重複 forward/back substitution。

L 的乘數是「消去時做了什麼」，不是最後的原始資料。

需要交換時的 $PA = LU$

若 pivot 位置為零，消去時必須先交換 rows。把交換集中成 permutation matrix P，可寫成

$$PA = LU.$$

對可逆方陣，總可選出讓每一步 pivot 非零的列，再得到此分解。P 左乘 A 表示預先排列 A 的 rows。

不要直接把 P 與 L, U 的順序交換。$PA = LU$ 的解讀是：先排列 A，再做沒有交換的 LU 消去。

有限精度演算法通常連「不為零」都不夠，而會選欄中絕對值最大者作為 pivot。

例如

$$A = \begin{bmatrix} 0 & 2 \\ 1 & 3 \end{bmatrix}, \quad b = \begin{bmatrix} 4 \\ 7 \end{bmatrix}, \quad P = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}.$$

先交換兩個 rows：$PA = \begin{bmatrix} 1 & 3 \\ 0 & 2 \end{bmatrix} = U$，因此 $L = I$。右端也交換成 $Pb = (7, 4)^T$；回代給 $y = 2, x = 1$，原式驗算為 $(4, 7)^T$。不能只交換 A 而保留原來的 b。多步分解在後期交換 rows 時，也須交換已存入 L 的舊乘數位置；否則即使 U 正確，$PA = LU$ 仍會失效。

換列時，已記錄的乘數也要跟著移動

令

$$A = \begin{bmatrix} 2 & 1 & 1 \\ 4 & 2 & 3 \\ 6 & 4 & 5 \end{bmatrix}, \qquad b = \begin{bmatrix} 3 \\ 8 \\ 12 \end{bmatrix}.$$

第一個主元為 2，乘數是 $\ell_{21} = 2$、$\ell_{31} = 3$。消去後，第二個主元位置為 0，但其下方是 1；交換 row 2 與 row 3，不能因此就判定矩陣奇異。



同一例子的 PA=LU 與求解

$$P=\begin{bmatrix}1&0&0\\0&0&1\\0&1&0\end{bmatrix},\quad L=\begin{bmatrix}1&0&0\\3&1&0\\2&0&1\end{bmatrix},\quad U=\begin{bmatrix}2&1&1\\0&1&2\\0&0&1\end{bmatrix}.$$

直接相乘得到 $LU=PA=\begin{bmatrix}2&1&1\\6&4&5\\4&2&3\end{bmatrix}$。由 $Ly=Pb=(3,12,8)^T$ 得 $y=(3,3,2)^T$，再由 $Ux=y$ 回代得 $x=(1,-1,2)^T$。代回原式可驗得 $Ax=b$。

若只換 U 的 row 卻把 L 的舊乘數留在原位，乘回的矩陣就不是 PA。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

可逆不代表可以省略換列

同一個例子有 $\det A=-2\neq 0$，但其左上 2×2 主子矩陣行列式是 $2\cdot 2-1\cdot 4=0$。因此矩陣可逆，仍可能在不換列的消去過程遇到零主元。

這裡交換一次 row，$\det P=-1$；由 $PA=LU$、$\det L=1$、$\det U=2$，可核對 $\det A=-2$。換列改變行列式的符號，也改變方程排列，不改變原方程組的解。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

成本與使用時機

對 dense $n \times n$ 方陣：

- Gaussian elimination 約需 $O(n^3)$ 次算術。
- forward/back substitution 約需 $O(n^2)$。
- 直接形成 inverse 後乘 b，還要額外做 matrix inverse，通常不划算。

所以常見策略是

$$A \longrightarrow PA = LU$$

一次，之後對多個 b 做 substitution。若只需要檢查某個系統的 consistency，elimination 比求 inverse 更直接。

精確代數可考慮 determinants 或 symbolic inverse，但一般數值計算仍以 factorized solve 為主。

成本的來源可直接求和：第 k 個 pivot 需更新約 $(n-k)^2$ 個尾端元素，每次一乘一減，共

$$2\sum_{k=1}^{n-1}(n-k)^2 = \frac{n(n-1)(2n-1)}{3} = \frac{2}{3}n^3 + O(n^2)$$

次浮點加乘；乘數除法與右端更新只增加 $O(n^2)$。三角回代只需處理 $n(n-1)/2$ 個非對角項，故為 $O(n^2)$。此模型計算算術次數，未包括資料搬移、稀疏填入或大整數位元成本。

不換列 LU 的條件與唯一性

對可逆方陣 A，存在不換列、每一步主元非零的 $A = LU$，若且唯若每個左上角主子矩陣 A_k 都可逆。若已有分解，因三角結構 $A_k = L_k U_k$，故

$$\det A_k = \prod_{j=1}^{k} u_{jj} \neq 0.$$

反向由消去歸納：第一個主元 $a_{11} = \det A_1 \neq 0$。前 $k-1$ 步已完成時，左上 k 階的行列式是前面主元乘上新主元；所以

$$u_{kk} = \frac{\det A_k}{\det A_{k-1}} \neq 0, \qquad \det A_0 = 1.$$

每一步都合法。此條件比「整個 A 可逆」更強；例如 $\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$ 可逆，卻需先交換 rows。

在固定的 P 下，若 $PA = LU = \widetilde{L}\widetilde{U}$，且兩個下三角因子的對角線皆為 1，則 $\widetilde{L}^{-1}L = \widetilde{U}U^{-1}$ 同時為單位下三角與上三角，只能是 I。所以這種正規化後的 L, U 唯一；不同選列規則的 P 不必相同。

Transpose 與 permutation

Transpose（轉置）定義為

$$(A^T)_{ij} = A_{ji}.$$

若 $A \in \mathbb{F}^{m\times n}$，則 $A^T \in \mathbb{F}^{n\times m}$。

Permutation matrix P 每列每欄恰有一個 1，其餘為 0。例：

$$P = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}.$$

PA 排列 A 的 rows，AP 排列 columns。若 P 是 permutation，則 $P^TP = I$。

P^T 就是 undo 的交換；交換不改變方程組的解集，只改變列順序。

Inner product 與 outer product

對 $x, y \in \mathbb{R}^n$：

$$x^Ty \in \mathbb{R}$$

是一個 inner product（內積），即純量。

若 $u \in \mathbb{R}^m$、$v \in \mathbb{R}^n$，則

$$uv^T \in \mathbb{R}^{m\times n},$$

稱為 outer product（外積型矩陣）。若 $u \neq 0$ 且 $v \neq 0$，它的第 j 個 column 是 $v_j u$，其中至少一欄非零，且全部落在 $\mathrm{span}\{u\}$，因此 rank 為 1；若 $u = 0$ 或 $v = 0$，所有 entries 都是 0，rank 反而是 0。

例：

$$\begin{bmatrix} 1 \\ 2 \end{bmatrix} \begin{bmatrix} 3 & -1 \end{bmatrix} = \begin{bmatrix} 3 & -1 \\ 6 & -2 \end{bmatrix},$$

第二個 row 是第一個 row 的 2 倍，rank 為 1。

若把 u 改成零向量，

$$\begin{bmatrix} 0 \\ 0 \end{bmatrix} \begin{bmatrix} 3 & -1 \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix},$$

則沒有非零 column 可張成 image，$\mathrm{rank} = 0$。同理，$v = 0$ 也得到零矩陣。因此

Inner product 與 outer product （續）

$$\operatorname{rank}(uv^T) = \begin{cases} 1, & u \neq 0 \text{ 且 } v \neq 0, \\ 0, & u = 0 \text{ 或 } v = 0. \end{cases}$$

u ≠ 0 ∧ v ≠ 0

u = (1,2)ᵀ, v = (3,−1)ᵀ

uvᵀ = [[3,−1],[6,−2]]

rank(uvᵀ) = 1

u = 0 ∨ v = 0

u = (0,0)ᵀ, v = (3,−1)ᵀ

uvᵀ = [[0,0],[0,0]]

rank(uvᵀ) = 0

$x^T y$ 與 xy^T 不只是左右順序不同，輸出型別也完全不同。

章末檢查

必須能：

- 以 rows 與 columns 解釋同一個 $Ax = b$。
- 從 pivot 與 free variable 判定無解、唯一解與無限多解。
- 寫出 row operation 對應的 elementary matrix。
- 說明 left multiplication 與 right multiplication 的差異。
- 判斷 inverse 的成立條件。
- 從 multipliers 重建 L，必要時寫 $PA = LU$。
- 分清 $x^T y$ 與 xy^T。

章末練習的重點是保留每一步的前提，尤其是「在哪個體上」與「是否需要 row exchange」。

下一章將以 column space、nullspace 與 basis 解釋本章的 pivot 結構。

延伸：積分內積的 Gram 系統

先備概念：內積、正交投影與定積分。

以下把消去法延伸到函數空間。取實連續函數空間 $C([-1,1],\mathbb{R})$，定義

$$\langle f,g\rangle=\int_{-1}^{1}f(t)g(t)\,dt.$$

連續性保證積分存在，且非零連續函數在某個小區間上絕對值有正下界，所以 $\int f^2>0$，這才給出內積的正定性。若容許任意逐點函數，「只在一點非零」的函數積分平方仍為零；不能直接在該集合上宣稱這是內積。使用 L^2 時則須把幾乎處處相等的函數視為同一個向量。

取不完全正交的基底 $p_1=1$、$p_2=1+t$。Gram matrix 的每個 entry 是 $G_{ij}=\langle p_i,p_j\rangle$：

$$G=\begin{bmatrix}\int 1 & \int(1+t)\\ \int(1+t) & \int(1+t)^2\end{bmatrix}=\begin{bmatrix}2 & 2\\ 2 & 8/3\end{bmatrix}.$$

若 $f=1+3t$，right-hand side 是

$$b=\begin{bmatrix}\langle p_1,f\rangle\\ \langle p_2,f\rangle\end{bmatrix}=\begin{bmatrix}2\\ 4\end{bmatrix}.$$

要求 $f=c_1p_1+c_2p_2$，不必猜係數；正交投影與完整展開都化為 Gram system

$$G\begin{bmatrix}c_1\\ c_2\end{bmatrix}=b.$$

延伸：積分內積的 Gram 系統 （續）

第一個方程給 $c_1+c_2=1$，第二個方程在代入後給 $c_2=3$，所以 $c_1=-2$。回代：

$$-2(1)+3(1+t)=1+3t.$$

$\det G=2(8/3)-2\cdot 2=4/3>0$，所以係數唯一。更一般地，$c^TGc=\|\sum_i c_ip_i\|^2$；函數獨立時它正定。若函數相依，$G$ 可能奇異，但由內積產生的右端 $b_i=\langle p_i,f\rangle$ 仍在 $\mathrm{Col}(G)$：對每個 $z\in\ker G$，$\sum_i z_ip_i=0$，故 $z^Tb=0$。因此投影存在；係數可不唯一，而所表示的投影函數唯一。f 不在 span 時只會有非零正交殘差，不會使這個 Gram 系統無解。

在 $C([0,2\pi],\mathbb{R})$ 使用相同的積分內積。若改用正交 basis $\{1,\cos x,\sin x\}$，投影係數可用除法直接得到。以 $f(x)=1+x$、$x\in[0,2\pi]$ 為例：

$$a_0=\frac{\int_0^{2\pi}(1+x)\,dx}{2\pi}=1+\pi,$$

$$a_1=\frac{\langle\cos x,f\rangle}{\langle\cos x,\cos x\rangle}=0,\qquad b_1=\frac{\langle\sin x,f\rangle}{\langle\sin x,\sin x\rangle}=-2.$$

所以有限 Fourier 投影是 $f_1(x)=1+\pi-2\sin x$。殘差 $e=x-\pi+2\sin x$ 與 1、$\cos x$、$\sin x$ 都正交；這正是有限維 least-squares projection，而不是把任意係數代入就算完成。

延伸：帶參數 Toeplitz 矩陣

先備概念：行列式、遞迴與特徵值。

對 $a \in \mathbb{R}$、正整數 n，再考慮

$$T_n(a) = \begin{bmatrix} a & 1 & & 0 \\ 1 & a & \ddots & \\ & \ddots & \ddots & 1 \\ 0 & & 1 & a \end{bmatrix}.$$

令 $D_n = \det T_n(a)$，並定義 $D_0 = 1$。沿第一 row 展開：

$$D_n = aD_{n-1} - D_{n-2}.$$

第二項來自第一 row 的 1 與下一 row 的 1，其結構正好是 D_{n-2}；此遞迴適用於 $n \geq 2$，配合 $D_0 = 1, D_1 = a$ 即可手算。前三項是

$$D_1 = a, \qquad D_2 = a^2 - 1, \qquad D_3 = a(a^2-1) - a = a(a^2-2).$$

也可用 eigenvectors。對 $k = 1, \dots, n$，令 $\theta_k = k\pi/(n+1)$、$(v_k)_j = \sin(j\theta_k)$，並把 boundary 值設為 $v_0 = v_{n+1} = 0$。三角恆等式給

$$(v_k)_{j-1} + (v_k)_{j+1} = 2\cos(\theta_k)(v_k)_j,$$

故 $T_n(a)v_k = (a + 2\cos\theta_k)v_k$。這些 vectors 非零且對應不同 eigenvalues，構成獨立集合；因此

$$D_n(a) = \prod_{k=1}^{n} (a + 2\cos(k\pi/(n+1))).$$

延伸：帶參數 Toeplitz 矩陣 （續）

$n = 3$ 時 eigenvalues 是 $a + \sqrt{2}, a, a - \sqrt{2}$，乘積回 $a(a^2-2)$。在 $\mathbb{R}$ 上 $T_3(a)$ 可逆若且唯若 $a \notin \{-\sqrt{2}, 0, \sqrt{2}\}$。這類題必須把 parameter 代入 determinant 的一般結構，不能只選一個數值算完就宣稱參數結論。

從消去到解集

先檢查矛盾列與自由變數，再選擇消去、回代或因子分解；換列時同步處理右端與已保存的乘數。

參考資料與署名

Lin Cing，《線性代數》手寫筆記。 Gilbert Strang, Introduction to Linear Algebra, 5th ed., Chapter 2.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study